

Reviewer Pack — Resultant Degree Exponential Lower Bound for ACC⁰ Circuit S...

Entry 3e977f2b8d85 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 12:07:54 UTC

Resultant Degree Exponential Lower Bound for ACC⁰ Circuit Size

Entry ID: 3e977f2b8d85 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 12:07:54 UTC

1. Conjecture

Field A (mathematical branch): Resultant Theory **Field B** (complexity object): ACC⁰ Circuit Size

Statement:

For any CNF formula with n variables, the degree of the resultant of its clause polynomials is at least $2^{\Omega(n)}$, implying that any ACC⁰ circuit computing this resultant must have size $2^{\Omega(n)}$.

Rationale (proposer's reasoning):

Resultants encode the solvability of systems of equations, and their degree reflects the complexity of the solution space. If ACC⁰ circuits cannot compute resultants efficiently, this would imply a separation between ACC⁰ and NEXP.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9f526dc082b1b2b2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            clauses.append(clause)
        return clauses

    def polynomial_from_clause(clause):
        x = Fraction(1)
        for literal in clause:
            x *= (Fraction(literal) + Fraction(abs(literal)))
        return x

    def resultant(poly1, poly2):
        if not poly1 or not poly2:
            return 0
        n = len(poly1)
        m = len(poly2)
        result = 0
        for i in range(n):
            for j in range(m):
                if poly1[i] != 0 and poly2[j] != 0:
                    result += poly1[i] * poly2[j]
        return result

    def degree_of_polynomial(poly):
        max_degree = 0
        for term in poly:
            degree = sum(abs(lit) for lit in term)
            if degree > max_degree:
                max_degree = degree
        return max_degree

    n = random.choice([5, 10, 15, 20, 30, 40])
    cnf = generate_cnf(n)
    polynomials = [polynomial_from_clause(clause) for clause in cnf]

    resultant_poly = polynomials[0]
```

```

for poly in polynomials[1:]:
    resultant_poly = resultant(resultant_poly, poly)

degree = degree_of_polynomial([resultant_poly])

metric_value = 2 ** (n / math.log(2))
conjecture_holds = degree >= metric_value
counterexample = "" if conjecture_holds else f"Resultant degree {degree} < 2^{n/2}"

return {
    "metric_name": "resultant_degree",
    "metric_value": degree,
    "instances_tested": n,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_86806a8c.py", line 81, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_86806a8c.py", line 62, in run_trial

```

    degree = degree_of_polynomial([resultant_poly])
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_86806a8c.py", line 49, in degree_of_polynomial

```

    degree = sum(abs(lit) for lit in term)
    ~~~~~

```

TypeError: 'int' object is not iterable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with TypeError, preventing data collection. The error suggests a fundamental issue with polynomial representation. | next: Debug polynomial construction in test_86806a8c.py to handle resultant_poly as a polynomial object rather than an integer

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111426
2	propose	ollama_remote	qwen3:8b	0	0	89212
3	preregistration	ollama_remote	qwen3:8b	0	0	25027
4	novelty	ollama_remote	qwen3:8b	0	0	21128
5	novelty	ollama_remote	qwen3:8b	0	0	16196
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17907
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8790
8	judge	ollama_remote	qwen3:8b	0	0	19094

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 308780 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/3e977f2b8d85.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3e977f2b8d85.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3e977f2b8d85.tar.gz` (if generated)